2411400-5
Aditya Pratap Singh Bhadoria

CSC-301 DBMS Assignment-3

Q.1 (a) and (b) are logically equivalent, but (c) can produce a different answer.

(a) → $\pi_{A,C}(R \bowtie \sigma_{B=1} S)$: $\bowtie$ matches tuples where R.B = S.B. Because $\sigma_{B=1}$ restricts tuples where B = 1, the join also forces R to match tuple B = 1.

(b) → $\pi_A(\sigma_{B=1} R) \times \pi_C(\sigma_{B=1} S)$: Filters both relations to B = 1, before taking cartesian product. Because B = 1, same constant in both sets, the product acts as an equijoin on B. So, equivalent as (a).

(c) → $\pi_{A,C}(\pi_A R \times \sigma_{B=1} S)$: By projecting R to only attribute A before any filtering happens, the B attribute is stripped away entirely. The product will then pair all values of A from R (including with B ≠ 1) with the C values from S where B = 1.

Ex ⇒ Let R(A,B) = {(5,2)} & S(B,C) = {(1,9)}

(a) $\sigma_{B=1} S$ gives {(1,9)}. R ⋈ is empty as R has no B = 1. result = ϕ.

(b) $\sigma_{B=1} R$ is empty. Empty × ( ) = ϕ.

(c) $\pi_A R$ = {(5)}, $\sigma_{B=1} S$ = {(1,9)}. So product $\pi_A R \times \sigma_{B=1} S$ Result = {(5,9)}.

Scanned with OKEN Scanner

Q.2) R has r tuples, S has s tuples ($r > 0, s > 0$)

(a) $R \cup \rho_{S(A,B)} S$

Min : max (r, s) - if all tuples of the smaller relation are identical to tuples in larger, duplicate ~~elimination~~ elimination collapses them.

Max : r + s - If both relation are entirely disjoint set of tuples.

(b) $\pi_{A,C}(R \bowtie S)$

Min : 0 (if there is no matching values of the shared attribute B b/w R and S)

Max : r × s (if every tuple in R has exact same B value and every tuple in S has exact same B value, turning into a full cartesian products, assuming all resulting A, C pairs are distinct.)

(c) $\pi_B R - (\pi_B R - \pi_B S)$ (equivalent to $\pi_B R \cap \pi_B S$)

Min : 0 (no overlapping B in two relations)

Max : min (r, s) (occurs if all B values are unique in there respective relation, and all B values from the smaller relation exists perfectly with the larger one.

(d) $(R \bowtie R) - R$

Min : 0
Max : 0

(The natural join of any relation with itself is itself, then R - R is empty set)

Scanned with OKEN Scanner

(e) $\sigma_{A>B} R \cup \sigma_{A<B} R$

Min: 0 (if every single row in R has A equal to B)

Max: ∞ (if no rows in R have A equal to B)

Q.3) a. $\pi_{pizzeria}(\sigma_{age<18}(Person) \bowtie Frequents)$

b. $\pi_{name}(\sigma_{gender='female'}(Person) \bowtie \sigma_{pizza='mushroom' \vee 'pepperoni'}(Eats))$

c. $(\pi_{name}(\sigma_{pizza='mushroom'}(Eats)) \cap \pi_{name}(\sigma_{pizza='pepperoni'}(Eats))) \bowtie \pi_{name}(\sigma_{gender='female'}(Person))$

d. $\pi_{pizzeria}(\pi_{pizza}(\sigma_{name='Amy'}(Eats) \bowtie \sigma_{price<10.00}(Serves)))$

e. if $PF = \pi_{pizzeria}(\sigma_{gender='female'}(Person) \bowtie Frequents)$

$PM = \pi_{pizzeria}(\sigma_{gender='male'}(Person) \bowtie Frequents)$

So, $(PF - PM) \cup (PM - PF)$

f. $Eats - \pi_{name, pizza}(Frequents \bowtie Serves)$

Q.4) 1.

```
SELECT DISTINCT sname
FROM Student
WHERE snum NOT IN (
    SELECT snum
    FROM Enrolled
);
```

Scanned with OKEN Scanner

2.
```
SELECT MAX(age)
FROM Student
WHERE major = 'HISTORY' OR Snum IN (
    SELECT E.Snum
    FROM Enrolled E
    JOIN Class C ON E.cname = C.name
    JOIN Faculty F ON C.fid = F.fid
    Where F.fname = 'I. Teach'
);
```

3.
```
SELECT name
FROM Class
WHERE room = 'R128'
UNION
SELECT cname
FROM Enrolled
GROUP BY cname
HAVING COUNT(Snum) >= 5;
```

4.
```
SELECT DISTINCT S.sname
FROM Student S
JOIN Enrolled E1 ON S.Snum = E1.Snum
Join Class C1 ON E1.cname = C1.name
JOIN Enrolled E2 ON S.Snum = E2.Snum
Join Class C2 ON E2.cname = C2.name
WHERE C1.meets_at = C2.meets_at AND C1.name <> C2.name;
```

Scanned with OKEN Scanner

5.
```
SELECT F.fname
FROM Faculty F
WHERE NOT EXISTS (
    SELECT room
    FROM Class
    EXCEPT
    SELECT C.room
    FROM Class C
    WHERE C.fid = F.fid
);
```

6.
```
SELECT F.fname
FROM Faculty F
WHERE 5 > (
    SELECT COUNT(E.snum)
    FROM Class C
    JOIN Enrolled E ON C.name = E.cname
    WHERE C.fid = F.fid
);
```

7.
```
SELECT level, AVG(age) AS avg_age
FROM Student
GROUP BY level;
```

8.
```
SELECT level, AVG(age) AS avg_age
FROM Student
WHERE level <> 'JR'
GROUP BY level;
```

9.
```
SELECT F.fname, COUNT(C.name) AS total_classes
FROM Faculty F
JOIN Class C ON F.fid = C.fid
WHERE F.fid NOT IN (
    SELECT fid
    FROM Class
    WHERE room <> 'R128'
)
GROUP BY F.fid, F.fname;
```

Scanned with OKEN Scanner

10.

```
SELECT S.Sname
FROM Student S
JOIN Enrolled E ON S.Ssnum = E.Snum
GROUP BY S.Ssnum, S.Sname
HAVING COUNT(E.Cname) >=ALL (
    SELECT COUNT(Cname)
    FROM Enrolled
    GROUP BY Snum
);
```

Scanned with OKEN Scanner